

1 Introduction to Computer Architecture

Computer Architecture refers to the design and organization of a computer's hardware components and how they interact to execute instructions efficiently. It encompasses the structure of the CPU, memory systems, input/output mechanisms, and the instruction set that defines how software communicates with hardware. At its core, it's about balancing performance, power consumption, cost, and complexity.

1.1 Key Principles

- **Performance Metrics:** Measured by throughput (instructions per second), latency (time per instruction), and benchmarks (e.g., SPEC, TPC).
- **Trade-offs:** Amdahl's Law (speedup limited by non-parallelizable parts), Moore's Law (transistor density doubles ~every 2 years, slowing post-2020), and Dennard Scaling (power density constant with scaling, broken ~2006 leading to multi-core era).
- **Levels of Abstraction:** From gates/transistors (logic level) to system-level (processors, memory, peripherals).
- **Importance:** Underpins all computing devices—from smartphones to supercomputers. In 2025, it drives AI hardware, quantum integration, and sustainable computing.

Computer architecture evolved from von Neumann's stored-program concept (1945), influencing modern designs.

2 History of Computer Architecture

- **1940s–1950s:** Early computers like ENIAC (1945, vacuum tubes, no stored programs). Von Neumann architecture (1945): Shared memory for data/instructions.
- **1960s:** Transistors enable smaller machines. IBM System/360 (1964): First family with compatible ISAs. Introduction of caches and pipelining.
- **1970s:** Microprocessors emerge (Intel 4004, 1971). Minicomputers and RISC ideas (Reduced Instruction Set Computing).
- **1980s:** Personal computers (IBM PC, 1981). RISC vs. CISC debate; Berkeley RISC project (1980), leading to MIPS.
- **1990s:** Superscalar processors (Pentium, 1993). Multi-core hints; PowerPC alliance.
- **2000s:** Multi-core era (Intel Core Duo, 2006) due to power walls. GPUs for parallel computing (NVIDIA CUDA, 2006).
- **2010s:** Mobile dominance (ARM architectures). Heterogeneous computing (CPU+GPU). Big.LITTLE (2011) for efficiency.
- **2020s:** AI accelerators (Google TPU, 2016+). RISC-V open standard gains traction. Apple Silicon (M1, 2020) shifts to ARM. Quantum and neuromorphic prototypes.
- **As of 2025:** Focus on AI-integrated architectures, neuromorphic chips mimicking brains for efficiency, quantum processors (e.g., Google's Willow with 105 qubits), and hybrid systems combining classical/quantum/edge computing. Energy-efficient designs address sustainability, with trends like in-memory computing and specialized AI hardware.

3 Fundamental Architectures

3.1 Von Neumann Architecture

- **Description:** Single memory for instructions and data; bottleneck (von Neumann bottleneck) due to sequential access.
- **Components:** CPU (ALU, control unit, registers), memory, I/O.
- **Pros/Cons:** Simple; limits parallelism.

3.2 Harvard Architecture

- **Description:** Separate memories for instructions (read-only) and data (read-write).
- **Pros:** Parallel fetch/execute; used in DSPs and embedded systems.
- **Modified Harvard:** Caches unify for flexibility (most modern CPUs).

3.3 Other Models

- **Dataflow:** Execution based on data availability (e.g., in GPUs).
- **Systolic Arrays:** Pipeline for matrix ops (e.g., TPUs).

4 Central Processing Unit (CPU)

The CPU executes instructions.

4.1 Components

- **Arithmetic Logic Unit (ALU):** Performs ops (add, AND, etc.).
- **Control Unit:** Fetches/decodes instructions, controls data flow.
- **Registers:** Fast storage (e.g., accumulator, program counter (PC), instruction register (IR)).
- **Clock:** Synchronizes operations (GHz speeds).

4.2 Instruction Cycle

1. Fetch: Load instruction from memory to IR.
2. Decode: Interpret opcode/operands.
3. Execute: ALU processes.
4. Store: Write results.

4.3 Enhancements

- **Pipelining:** Overlap stages (fetch, decode, execute). 5-stage classic (IF, ID, EX, MEM, WB). Hazards: Structural (resource conflict), data (dependencies), control (branches).
 - Solutions: Forwarding, stalling, branch prediction.
- **Superscalar:** Multiple pipelines (e.g., issue 4 instructions/cycle).
- **Out-of-Order Execution:** Reorder instructions dynamically (Tomasulo's algorithm).
- **Speculative Execution:** Guess branches (e.g., Meltdown/Spectre vulnerabilities exposed this).

5 Instruction Set Architectures (ISA)

Defines machine code format.

5.1 Types

- **CISC (Complex):** Rich instructions (e.g., x86: variable length, many addressing modes). Pros: Dense code; Cons: Complex decoding.
- **RISC (Reduced):** Simple, fixed-length (e.g., ARM, RISC-V: load/store, few ops). Pros: Faster pipelining; Cons: More instructions needed.
- **VLW/EPIC:** Compiler packs instructions (e.g., Itanium).

5.2 Examples

ISA	Characteristics	Used In
x86/x64	CISC, backward compatible	Intel/AMD PCs, servers
ARM	RISC, power-efficient	Mobiles, Apple M-series
RISC-V	Open-source RISC	Embedded, emerging servers
MIPS	RISC, educational	Older routers, PlayStation

Table 1: Instruction Set Architectures

- **Extensions:** SIMD (e.g., AVX for vector ops), virtualization (VT-x).

6 Memory Hierarchy

Optimizes speed/cost using locality (temporal/spatial).

6.1 Levels

- **Registers:** Fastest, smallest (bytes).
- **Cache:** SRAM, levels (L1: per-core, fast; L2/L3: shared). Policies: Write-back/through, replacement (LRU), associativity (direct-mapped, set-associative).
- **Main Memory (RAM):** DRAM, volatile.
- **Secondary Storage:** SSD/HDD, persistent.
- **Virtual Memory:** OS-managed, uses paging (page tables, TLB for fast translation). Swapping for oversubscription.

6.2 Advanced

- **NUMA (Non-Uniform Memory Access):** In multi-socket systems, local/remote memory.
- **In-Memory Computing:** Process data in RAM to reduce movement (e.g., for AI workloads in 2025).

7 Input/Output and Peripherals

- **I/O Interfaces:** Buses (PCIe for high-speed), ports (USB, HDMI).
- **DMA:** Direct memory access bypasses CPU.
- **Interrupts:** Hardware signals for events (vs. polling).

8 Parallelism and Multi-Core Architectures

- **Types:**
 - **Instruction-Level (ILP):** Pipelining, superscalar.
 - **Thread-Level:** Multi-core (SMP: Symmetric Multi-Processing).
 - **Data-Level:** SIMD (e.g., vector units).
- **Multi-Core:** Cores share caches/bus. Coherence: MESI protocol.
- **GPUs:** Thousands of cores for parallel tasks (e.g., CUDA cores).
- **Heterogeneous:** CPU + accelerators (e.g., AMD APUs).
- **Challenges:** Amdahl's Law, synchronization (locks, atomics).

9 Advanced and Modern Topics

- **Branch Prediction:** Static (compiler) vs. dynamic (history tables, neural predictors in 2020s CPUs).
- **Power Management:** DVFS (Dynamic Voltage/Frequency Scaling), dark silicon (not all transistors active).
- **Security Features:** ASLR, NX bits, SGX (enclaves).
- **AI Accelerators:** NPUs (Neural Processing Units) in SoCs (e.g., Apple Neural Engine).
- **In 2025 Advancements:**
 - **Neuromorphic Computing:** Brain-inspired chips (e.g., IBM TrueNorth evolutions) for low-power AI, processing in parallel like neurons.
 - **Quantum Integration:** Hybrid classical-quantum systems; Google's Willow solves complex problems rapidly.
 - **Energy-Efficient Architectures:** Green designs with specialized cores, in-memory processing, and SLMs deployable on edge hardware for reduced latency/carbon footprint.

- **Co-Design:** AI models and hardware developed together; programmable FPGAs for adaptable pipelines.
- **Hybrid Systems:** Combine CPU, edge, quantum, neuromorphic for optimized workloads.

10 Emerging Architectures

- **Quantum:** Qubits, superposition/entanglement. Architectures: Gate-based (IBM Q), annealing (D-Wave).
- **Neuromorphic:** Spiking neural networks (e.g., Intel Loihi).
- **Optical/Photonic:** Light-based for faster interconnects.
- **DNA/Biological:** Molecular storage/computing (experimental).
- **Challenges:** Scalability, error correction (quantum), power in data centers.

11 Design Tools and Methodologies

- **HDLs:** Verilog/VHDL for RTL design.
- **Simulation:** ModelSim, Vivado.
- **Synthesis:** To gates/FPGAs.
- **Metrics:** CPI (Cycles Per Instruction), MIPS/FLOPS.

12 Applications and Case Studies

- **Embedded:** ARM in IoT.
- **High-Performance Computing (HPC):** Exascale systems (e.g., Frontier, 2022+).
- **Mobile:** Big.LITTLE for battery life.
- **Servers:** AMD EPYC with chiplets.

13 Best Practices

- **Optimization:** Profile for bottlenecks (e.g., cache misses).
- **Scalability:** Design for parallelism.
- **Sustainability:** Prioritize low-power architectures in 2025.